

COMP1531

Software Engineering Fundamentals

Week 5

Full-Stack - HTTP Servers (Part II)

In This Lecture

— — —

Why?

- Web servers are fundamental part of web-based full-stack software

What?

- ~~Networks~~
- ~~Express Server~~
- APIs
- CRUD Operations

NodeJS Express Server

— — —

A very popular npm library exists to allow you to run your own HTTP server with NodeJS. It's called [Express Server](#).

Let's install it (of course)

- **`npm install express cors morgan`**
 - [cors](#) to allow access from other domains (needed for frontend to connect)
 - [morgan](#) (OPTIONAL) to log (print to terminal) incoming HTTP requests.

Install the type definitions for the dependencies

- **`npm install --save-dev @types/express@4.17.21 @types/cors @types/morgan`**

Now we can run our own HTTP server!

NodeJS Express Server

— — —

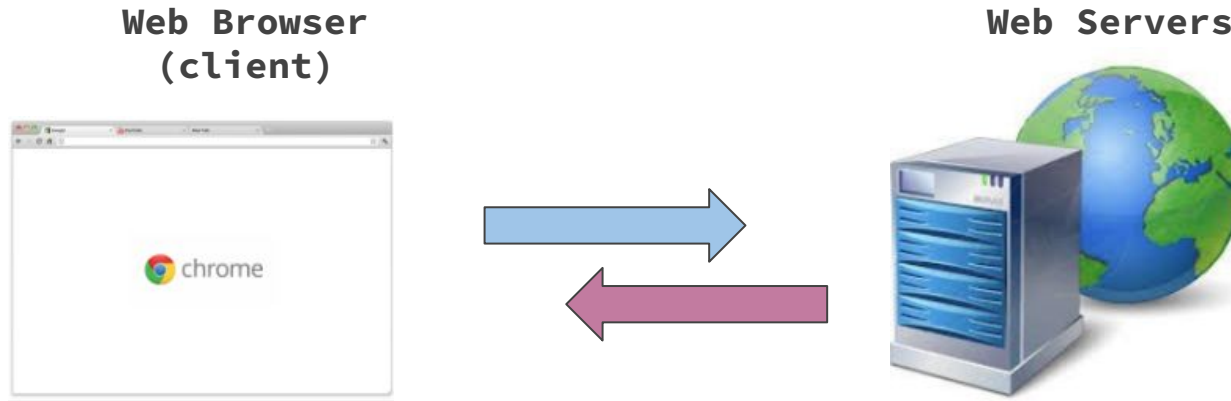
express_server.js

```
1 // Import the Express framework from the 'express' package
2 import express from 'express';
3
4 // Create an Express application instance
5 const app = express();
6
7 // Define the port number the server will listen on
8 const port = 3000;
9
10 // Add built-in middleware to parse incoming requests with text payloads
11 app.use(express.text());
12
13 // Define a GET route for the path '/hello'
14 app.get('/hello', (req, res) => {
15   // Send a plain text response back to the client
16   res.send('Hello!');
17 });
18
19 // Define a GET route for the path '/whats/up'
20 app.get('/whats/up', (req, res) => {
21   // Send a JSON string as the response
22   res.send(JSON.stringify({
23     // Property named "value" with the string "not much"
24     value: 'not much',
25   }));
26 });
27
28 // Start the server and make it listen on the specified port
29 app.listen(port, () => {
30   // Log a message to the console once the server starts successfully
31   console.log(`Listening on port ${port}`);
32 });
```

HTTP

HTTP: Hypertext Transfer Protocol

i.e. Protocol for sending and receiving HTML documents (nowadays much more)



Web browsers are applications to request and receive HTTP.

What Servers Are Used For

— — —

In the past, servers were just used to serve files back to client's browsers.

Nowadays, they're used to quietly exchange large amounts of information back and forth between client and server behind the scenes. Often this happens whilst the server is acting as an API.

This is probably done by sending **JSON** instead of **text/HTML**.



— — —

JSON file structure that returns a list of **student records** — each containing a student's name, ID, major, and WAM

```
{
  "students": [
    {
      "id": "S1001",
      "name": "Alice Brown",
      "major": "Computer Science",
      "WAM": 82.5
    },
    {
      "id": "S1002",
      "name": "Liam Chen",
      "major": "Software Engineering",
      "WAM": 76.3
    },
    {
      "id": "S1003",
      "name": "Maya Patel",
      "major": "Data Science",
      "WAM": 89.1
    }
  ]
}
```

Servers Often Respond With JSON

```
1 // Import the Express framework
2 import express from 'express';
3
4 // Create server.
5 const app = express();
6
7 // Define the port number the server will listen on
8 const port = 3001;
9
10 // Add middleware to automatically parse incoming JSON in request bodies
11 app.use(express.json());
12
13 // When someone visits http://localhost:3000/whats/up, this sends a JSON response
14 app.get('/whats/up', (req, res) => {
15   res.json({
16     value: 'not much',
17   });
18 });
19
20 // Start the server and listen on the specified port
21 app.listen(port, () => {
22   console.log(`Listening on port ${port}`);
23 });
```

We can send
javascript objects
as JSON

`express_server.js`

Path Parameters vs Query Parameters

— — —

Use QUERY parameters when:

- Filter, search, sort, or paginate
- Customize the response
- Usually **optional**
- Example: GET /users?page=2&sort=name

Use PATH parameters when:

- Identify a specific resource
- The value is required
- Changing value means different resource
- Example: GET /users/123

How Servers Receive Information

```
1 import express from 'express';
2
3 const app = express();
4 const port = 3000;
5
6 app.use(express.json());
7
8 app.get('/my/url', (req, res) => {
9   const name = req.query.name;
10  const age = req.query.age;
11  res.json({
12    name: `Name is ${name}`,
13    age: `Age is ${age}`,
14  });
15 });
16
17 app.listen(port, () => {
18   console.log(`Listening on port ${port}`);
19 });
```

express_server.js

Servers can receive information through the URL. This is called the **query** data.
Eg. Add ?name=Shaveen&age=22 to the end of our url.

How Servers Receive Information

```
1 import express from 'express';
2
3 const app = express();
4 const port = 3001;
5
6 app.use(express.json());
7
8 app.get('/my/url/:name', (req, res) => {
9   const name = req.params.name;
10  res.json({
11    name: `Name is ${name}`,
12  });
13 });
14 // app.get(...) tells your server:
15 // 🖱️ "When someone makes a GET request to a
16 // certain path (URL), here's how to respond."
17
18
19 app.listen(port, () => {
20   console.log(`Listening on port ${port}`);
21 });
```

input_params.ts

Servers can receive information through the URL. This is called the **params** data.

Eg. Try localhost:3000/my/url/Shaveen



API Clients To Talk To Servers

— — —

We can use API clients to talk to servers.

1. Locally you can install "Advanced Rest Client" or on the CSE machines you can run 1531 arc.

OR

2. Use an npm package **sync-request-curl** to allow us to programmatically send RESTful API requests



API Clients To Talk To Servers (ARC)

It brings up an app like this. URL: <https://install.advancedrestclient.com/home>

Alternatives To app.get

We actually have an alternative to `app.get` called **`app.post`** which is where we send information through a body and not over the URL.

- But we can't use the browser for this anymore...

```
1 import express from 'express';
2
3 const app = express();
4 const port = 3000;
5
6 app.use(express.json());
7
8 app.post('/hello', (req, res) => {
9   const name = req.body;
10   res.json({
11     name: `Name is ${name}`,
12   });
13 });
14
15 app.listen(port, () => {
16   console.log(`Listening on port ${port}`);
17 });
```

`post_example.ts`



Requests Made Via Code

We don't need an API client to send requests to servers, we can actually send requests via code



Requests Library

We can use an npm package `sync-request-curl` to allow us to programatically send RESTful API requests.

```
npm install sync-request-curl
```

We can send HTTP requests to our server



Requests Library

```
1 import request from 'sync-request-curl';
2
3 const res = request(
4   'POST',
5   'http://localhost:3000/books',
6   {
7     json: {
8       title: 'helloeveryone',
9       author: 1531,
10      year: 2025
11    },
12  }
13 );
14 const res1 = request(
15   'GET',
16   'http://localhost:3000/books',
17 );
18
19 console.log(JSON.parse(String(res.getBody())));
```

requests.ts



HTTP Crud Methods

— — —

get and post aren't the only types of requests we can send servers...



All The HTTP Methods

— — —

These are the remaining methods. They each have their own meaning about what the underlying action does.

Method	Operations
GET	Read
POST	Create
PUT	Update
DELETE	Remove

Common HTTP Codes

— — —

- 200 - OK
- 201 - Created
- 400 - Bad request
- 403 - Forbidden
- 404 - Not found
- 500 - Internal server error
- 502 - Bad gateway

... [and many more](#)

HTTP Error Codes

— — —

- 1xx informational response
 - The request was received, continuing process
- 2xx successful
 - The request was successfully received, understood, and accepted
- 3xx redirection
 - Further action needs to be taken in order to complete the request
- 4xx client error
 - The request contains bad syntax or cannot be fulfilled
- 5xx server error
 - The server failed to fulfil an apparently valid request

Lets try a CRUD example

— — —

`books_server.ts`



All The HTTP Methods

— — —

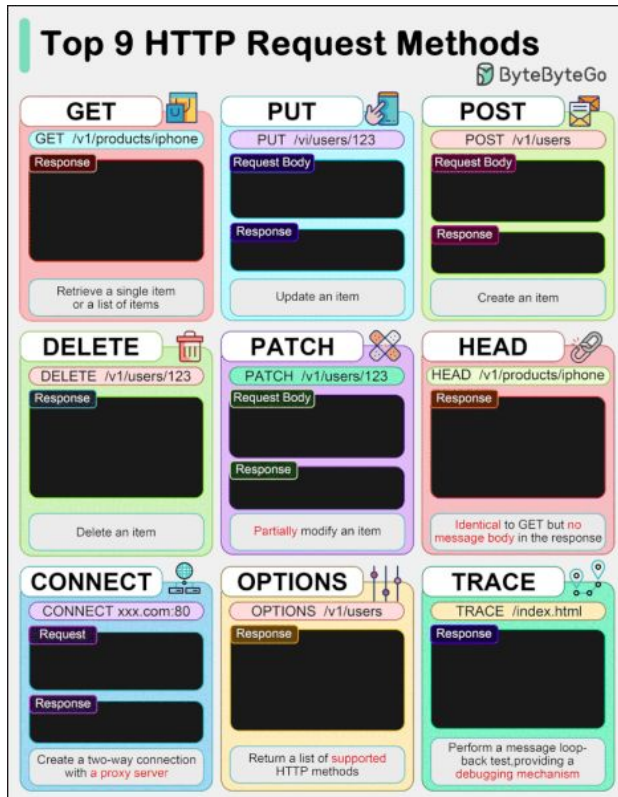
Method	Operations	req.params	req.query	req.body	Response
GET	Read	Yes	Yes	No	res.json()
POST	Create	Yes	No	Yes	res.json()
PUT	Update	Yes	No	Yes	res.json()
DELETE	Remove	Yes	Yes	No	res.json()



All The HTTP Methods

— — —

Well not quite all ...



[source](#)

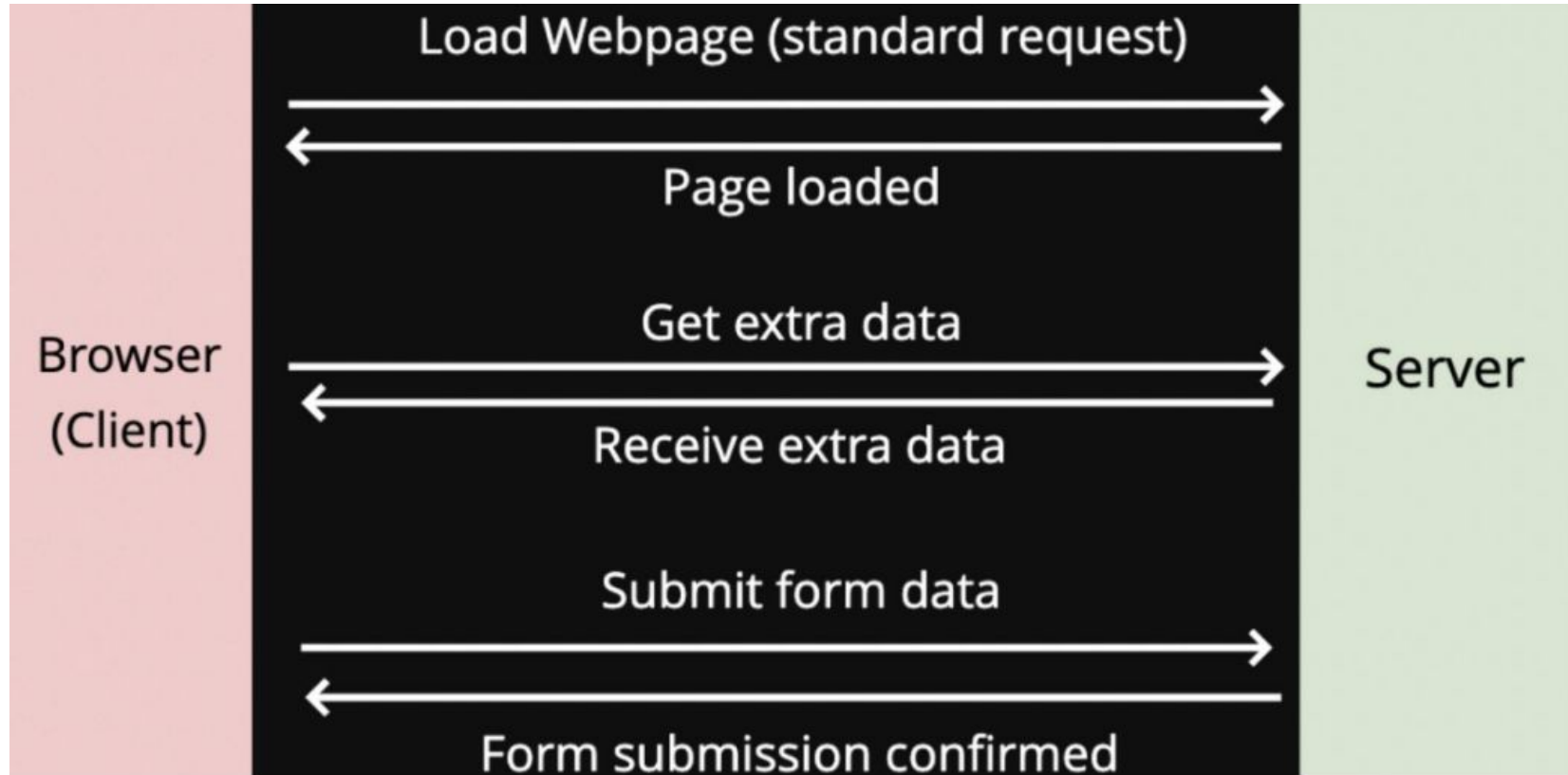
API

An API (Application Programming Interface) refers to an interface exposed by a particular piece of software.

The most common usage of "API" is for Web APIs, which refer to a "**contract**" that a particular service provides. The interface of the service acts as a black box and indicates that for particular **endpoints**, and given particular *input*, the client can expect to receive particular *output*.



Web API



RESTful API

— — —

A RESTful API follows the principles of **REST (Representational State Transfer)**

- Uses **URLs** to identify resources.
- Uses **HTTP methods** (**GET, POST, PUT, DELETE**) to perform operations.
- Exchanges data, usually in **JSON** format.
- Is **stateless**, meaning each request contains everything the server needs to handle it. Each request is **independent**.

Returning Errors

— — —

Restful APIs can also return errors

```
1 import express from 'express';
2
3 const app = express();
4 const port = 3001;
5
6 app.use(express.text());
7
8 app.get('/apple/:name', (req, res) => {
9   const name = req.params.name;
10   if (name === 'Hayden') {
11     res.status(400).json({
12       error: 'Bad name',
13     });
14   } else {
15     res.json({
16       msg: `Hi ${name}, thanks for sending apple!`,
17     });
18   }
19 });
20
21 app.listen(port, () => {
22   console.log(`Listening on port ${port}`);
23 });
```

error.ts

HTTP Error Codes

— — —

- 1xx informational response
 - The request was received, continuing process
- 2xx successful
 - The request was successfully received, understood, and accepted
- 3xx redirection
 - Further action needs to be taken in order to complete the request
- 4xx client error
 - The request contains bad syntax or cannot be fulfilled
- 5xx server error
 - The server failed to fulfil an apparently valid request

Common HTTP Codes

— — —

- 200 - OK
- 201 - Created
- 400 - Bad request
- 403 - Forbidden
- 404 - Not found
- 500 - Internal server error
- 502 - Bad gateway

... [and many more](#)

Wrapping into COMP1531 Project

— — —

- Iteration 2 requires that you implement an HTTP server
 - However, many routes are just Iteration 1 wrappers
- You can “wrap” iteration 1 in an HTTP server
 - Most of the "server" stuff you'll do is just routing, gathering bodies, and returning responses
 - Treat your iteration 1 functions like blackboxes.

Other Express Helpers

— — —

Sometimes we put version numbers in our routes to help us keep track, e.g.:

- `/v1/names/list`
- `/v2/names/list`
- `/v3/names/list`

You will work with this principle in your major project.

Other - Token Encoding

— — —

A token is generally stringified for sending over HTTP - since everything over an HTTP request needs to be stringified. This is typically done with JSON.

If you pass a JSONified object (as opposed to just a string or a number) as a token, we recommend that you use [encodeURIComponent](#) and [decodeURIComponent](#) to encode it to be friendly for transfer over URLs.

Feedback

— — —



Or go to the [form here](#)